URBANMOVE · DOCUMENTO DE TRABAJO

Reunión de arranque

Decisiones, reparto y plan de la primera semana

Equipo: 3 integrantes

Fecha: 11 de noviembre de 2026

Versión: 1.0

Fuente: docs/00-gestion/KICKOFF.md

Documento para la primera reunión del equipo. Las decisiones marcadas y el acta se actualizan en el Markdown; este PDF es una copia de trabajo.

Reunión de arranque — UrbanMove

> **Documento de trabajo para los 3 integrantes.**
> **Duración prevista:** 120 minutos.
> **Objetivo:** cerrar las decisiones que bloquean la Fase 0, confirmar el reparto y salir con tareas asignadas y fechadas.
> **Cómo se usa:** se lee antes de la reunión, se marca cada casilla durante la reunión y se archiva el acta al final. La versión PDF es una copia para llevar; la fuente de verdad es este Markdown.
> **Copia para imprimir:** KICKOFF.pdf.

---

1. Lectura previa (15 minutos)

Cada integrante llega habiendo leído, al menos:

- [ ] PLAN.md — fases, reparto y riesgos.
- [ ] GITHUB-WORKFLOW.md — issues, etiquetas y tablero.
- [ ] ARCHITECTURE.md — servicios y comunicación.
- [ ] SERVICE-ROADMAP.md — orden y dependencias.
- [ ] DATA-ARCHITECTURE.md — PostgreSQL, S3 y marts.
- [ ] CODE-STANDARDS.md — cómo se escribe el código.
- [ ] TESTING.md — qué se prueba y con qué.
- [ ] LOCAL-DEV.md — qué instalar y cómo arrancar.
- [ ] CONTAINERS.md — cómo se empaqueta.
- [ ] EVENTS.md — eventos, outbox e idempotencia.

Si alguien no ha podido leer todo, se lee como mínimo la sección 5 de este documento (decisiones) antes de empezar.

---

2. Cómo rellenar este documento

Este Markdown es la fuente de verdad. El PDF es solo una copia para imprimir y no se edita.

2.1. Editar desde GitHub (lo más rápido)

1. Abrir `docs/00-gestion/KICKOFF.md` en GitHub.
2. Pulsar el lápiz (**Edit this file**).
3. Hacer los cambios.
4. Elegir **Commit changes**:
   - directo a `main` si es una corrección pequeña;
   - en una rama `docs/reunion-<fecha>` y abrir un PR si son cambios grandes.
5. Confirmar el commit con un mensaje como `docs: update kickoff minutes`.

2.2. Editar en local (recomendado para la reunión)

```
git pull
# editar docs/00-gestion/KICKOFF.md en VS Code o IntelliJ
git add docs/00-gestion/KICKOFF.md
git commit -m "docs: update kickoff decisions and actions"
git push
```

Nadie más toca el archivo mientras se edita. Si dos personas lo cambian a la vez, el segundo hace `git pull --rebase` y resuelve el conflicto a mano; no se sobrescribe.

2.3. Reglas de marcado

Quiero...	Cómo se escribe
Marcar una tarea hecha	`- [ ]` → `- [x]`
Cerrar una decisión	Cambiar ☐ por ☑ en la columna **¿Cerrada?**
Rellenar un nombre	Sustituir `_____` por el nombre real
Anotar la decisión final	Escribirla en la tabla del **Acta** (sección 13.1)
Aplazar una decisión	Anotarla en **Decisiones aparcadas** con fecha de revisión
Añadir una acción	Añadir una fila en **Acciones acordadas** con dueño y fecha

No se borran las decisiones descartadas: se marcan como cerradas y se explica el motivo en el acta. El histórico se conserva.

2.4. Quién actualiza qué

Momento	Responsable	Acción
Antes de la reunión	A	Comprueba que el documento está en `main`
Durante la reunión	A	Marca casillas y rellena el acta
Después de la reunión	A	Commit con el acta y aviso al grupo
Cambio de decisión posterior	Quien la propone	Actualiza el documento y, si toca arquitectura, crea un ADR

2.5. El PDF

- No se edita a mano.
- Se regenera solo cuando cambie el Markdown y haga falta una copia para imprimir.
- Si no se regenera, el PDF queda desactualizado: **lo válido es siempre el Markdown**.
- Para regenerarlo hace falta `pandoc`:

```
pandoc docs/00-gestion/KICKOFF.md -o docs/00-gestion/KICKOFF.pdf --pdf-engine=wkhtmltopdf
```

Si no hay `pandoc` instalado, se pide al asistente que lo regenere o se imprime el Markdown desde GitHub.

---

3. Agenda (120 minutos)

Min	Tema	Resultado esperado
0-10	Contexto y objetivo del proyecto	Todos alineados en alcance y plazo
10-40	Decisiones clave (sección 5)	Cada decisión marcada o aparcada con fecha
40-55	Alcance: mínimos y recortes	Lista de lo que entra y lo que no
55-70	Reparto de responsabilidades	Nombres reales en A, B y C
70-85	Fase 0: tareas de la semana 1	Cada tarea con dueño y día
85-95	Riesgos y mitigaciones	Un responsable por riesgo
95-105	Rituales y definición de "hecho"	Aceptados sin discusión
105-115	Primeras acciones y fechas	10 acciones con fecha
115-120	Acta y cierre	Documento firmado por los 3

Regla de la reunión: si una decisión no se cierra en 3 minutos, se aplica la recomendación marcada y se revisa en la integración del viernes. No se bloquea la Fase 0.

---

4. Contexto en una página

Aspecto	Situación
Equipo	3 personas
Plazo	6-8 semanas
AWS	Incierto; se empieza en local
Interfaz	No prioritaria: Swagger/Postman + Grafana
Estado del repo	Solo documentación; cero código
Primer entregable	Fase 0: entorno + `zones` + Keycloak + evento en S3 + CI verde

Lo que se construye: 13 microservicios, 5 Lambdas, un bus de eventos con outbox y DLQ, un Data Lake en S3, 7 data marts y un dashboard.

Lo que no se construye: app móvil, pagos reales, mapas reales, machine learning avanzado, Kubernetes, alta disponibilidad multirregión.

---

5. Decisiones clave

Cada fila se marca durante la reunión. La columna **Recomendación** ya está decidida por defecto: si nadie presenta un argumento nuevo, se adopta tal cual.

5.1. Decisiones tecnológicas

ID	Decisión	Alternativas	Recomendación	Dueño	¿Cerrada?
D1	Versión de Java y Spring	Boot 4.0.9, Quarkus	**Java 21 + Spring Boot 3.5.16** (ecosistema y springdoc estables)	Todos	☐

ID	Decisión	Alternativas	Recomendación	Dueño	¿Cerrada?
D2	Estructura del repositorio	13 repos, submódulos	**Monorepo** con POM padre y `services/`	C	☐
D3	Estrategia de nube	AWS desde el día 1, solo local	**Local-first**: Compose + LocalStack; AWS después	Todos	☐
D4	Base de datos operacional	DynamoDB, MongoDB	**PostgreSQL 16**, una base por servicio + `keycloak_db`; igual en local (contenedor) y en AWS (RDS)	C	☐
D5	Topología RDS	1 instancia, 13 instancias, Aurora	**Dos instancias** (operacional y analítica) con plan B de una	C	☐
D6	Mensajería	Kafka/MSK, RabbitMQ, SNS	**EventBridge + SQS + DLQ + outbox**	Todos	☐
D7	Capa analítica	Batch + Lambda, Redshift, Athena	**Kappa + Medallón** en S3, un bucket con `raw/trusted/curated`	C	☐
D8	Data marts	Vistas al vuelo, otra BD	**Proyecciones en `analytics_db`**, reconstruibles desde eventos	C	☐
D9	Identidad	Cognito, login propio	**Keycloak 26.8.2** con `keycloak_db`; Cognito solo si hay AWS	A	☐
D10	Estilo de API	GraphQL, gRPC	**REST versionado `/api/v1`** + RFC 7807 + OpenAPI 3	Todos	☐
D11	Empaquetado	Buildpacks, Jib	**Dockerfile multi-stage** con JRE Alpine y usuario no root	B	☐
D12	Despliegue AWS	EKS, Beanstalk	**EC2 + Compose** o ECS Fargate si hay crédito; sin Kubernetes	B	☐
D13	Testing	H2, solo unitarios	**JUnit 5 + AssertJ + Testcontainers**; H2 prohibido	Todos	☐
D14	TDD	TDD estricto en todo, sin TDD	**TDD selectivo**: dominio, contratos e idempotencia	Todos	☐
D15	Frontend	Web propia, ninguna	**Sin frontend propio**: Swagger, colección de API y Grafana	Todos	☐
D16	Cliente de API	Postman, Bruno, curl	**Bruno** (colecciones de texto, diffs limpios en Git)	A	☐
D17	CI/CD	Jenkins, GitLab CI	**GitHub Actions + GHCR**	C	☐
D18	Idioma	Todo español, todo inglés	**Código e identificadores en inglés; documentación y commits en español**	Todos	☐
D19	Paquete raíz	`com.example`, `es.ucm`	**`com.urbanmove`**	Todos	☐
D20	IaC	CloudFormation, CDK, Terraform	**Terraform** cuando exista AWS; no bloquea la Fase 0	C	☐

5.2. Decisiones de alcance

ID	Decisión	Recomendación	¿Cerrada?
A1	Servicios obligatorios	Los 13 del catálogo; mínimo 10 para aprobar	☐
A2	Recorte si el plazo aprieta	Recortar Fases 3 y 4, nunca el flujo de usuario	☐
A3	DynamoDB para telemetría	Solo ampliación opcional; no camino crítico	☐
A4	Dashboard	Grafana con 6 paneles; no desarrollar UI	☐

ID	Decisión	Recomendación	¿Cerrada?
A5	Datos de demo	3 zonas, 10 vehículos, 3 usuarios, 20 viajes simulados	☐
A6	Autenticación en demo	Token de Keycloak en Postman/Bruno; sin formulario propio	☐

5.3. Decisiones de proceso

ID	Decisión	Recomendación	¿Cerrada?
P1	Ramas	`main` protegida; `feat/`, `fix/`, `docs/` de menos de 3 días	☐
P2	PR	Pequeño, con issue, checklist y CI verde; 1 revisión obligatoria	☐
P3	Contratos	OpenAPI y JSON Schema **antes** de implementar cada servicio	☐
P4	Integración	Miércoles parcial y viernes completa, sin excepciones	☐
P5	ADR	Toda decisión que afecte a más de un servicio se documenta	☐
P6	Secretos	Cero credenciales en el repo; `.env` ignorado	☐
P7	Revisión de código	Checklist de `CODE-STANDARDS.md` en cada PR	☐

> **Decisiones ya cerradas que no se reabren:** local-first (ADR 0001), PostgreSQL frente a DynamoDB (0002), Kappa + Medallón (0003), Keycloak frente a Cognito (0004), Grafana como dashboard (0005) y dos instancias RDS (0006).

---

6. Reparto del equipo

Rellenar con nombres reales. La propuesta es por dominio, no por tecnología.

Rol	Nombre	Dominio	Servicios y tareas
A	_____	Usuario	`identity`, `user-profiles`, `reservations`, `rides`, colección Bruno, pruebas de esos servicios
B	_____	Operaciones	`inventory`, `availability`, `telemetry`, `incidents`, `maintenance`, `relocation`, Lambdas de expiración y detección
C	_____	Datos y plataforma	`zones`, `history`, `analytics`, Data Lake, marts, Grafana, CI/CD, Compose, Terraform

Responsabilidades transversales:

- [ ] **Contratos:** A revisa REST, B revisa eventos, C valida esquemas.
- [ ] **Integración semanal:** C convoca y prepara el entorno.
- [ ] **Acta y tablero:** A mantiene las issues al día, si el equipo decide usarlas (ver GITHUB-WORKFLOW.md).
- [ ] **Demo:** B prepara el guion y los datos.
- [ ] **Informe y PDF:** los tres escriben su parte; C compila.

Reglas de convivencia:

- [ ] Nadie mergea sin revisión de otro.

- [ ] Nadie trabaja más de 3 días en una rama.
- [ ] Un servicio no lee la base de otro.
- [ ] Si algo bloquea más de 2 horas, se avisa al grupo; no se sufre en silencio.

---

7. Alcance confirmado

7.1. Mínimos que no se recortan

- [ ] 10 microservicios atómicos con base de datos propia.
- [ ] `history` como servicio independiente.
- [ ] Autenticación con roles (usuario, operaciones, administrador).
- [ ] Al menos un bus/cola con DLQ y outbox.
- [ ] 3 Lambdas conectadas a eventos reales.
- [ ] Data Lake `raw/trusted/curated`.
- [ ] Al menos 4 data marts.
- [ ] Recolocación generada desde la demanda.
- [ ] Dashboard con flota, reservas, viajes, mantenimiento y salud.
- [ ] 4 mecanismos de tolerancia a fallos y 2 fallos demostrados.
- [ ] Tests, imágenes, migraciones y CI documentados.
- [ ] Informe y defensa de los 3 integrantes.

7.2. Ampliaciones que se pueden recortar

- [ ] DynamoDB para telemetría.
- [ ] Recolocación automática avanzada.
- [ ] Frontend web propio.
- [ ] Machine learning sencillo.
- [ ] Despliegue ECS Fargate.

---

8. Fase 0: semana 1

Criterio de fin de semana: `docker compose --profile infra up` funciona, `zones` responde, Keycloak emite token, un evento llega a `raw/` en S3 y el CI está verde.

Día 1 — Repositorio y Compose

- [ ] Crear estructura `services/`, `lambdas/`, `infra/`, `contracts/`, `clients/`, `scripts/`.
- [ ] `.env.example` con valores de desarrollo (sin secretos reales).
- [ ] `docker-compose.yml` con PostgreSQL, Keycloak, LocalStack, Prometheus y Grafana.
- [ ] Script de creación de las 14 bases y sus usuarios.
- [ ] Healthchecks y volúmenes persistentes.
- [ ] Comprobar `docker compose ps` en `healthy`.
- [ ] Commit `chore: create repository skeleton for phase 0`.

Dueño: C. **Apoyo:** todos.

Día 2 — POM padre, esqueleto y realm

- [ ] POM padre con Java 21, Spring Boot 3.5.16, JaCoCo y surefire/failsafe.
- [ ] Esqueleto hexagonal de `zones` con `/actuator/health`.
- [ ] Realm `urbanmove` con roles `USER`, `OPERATOR`, `ADMIN` y 3 usuarios.
- [ ] Script de LocalStack: bucket, bus, colas, DLQ y reglas.
- [ ] Comprobar token con `grant_type=password`.
- [ ] Comprobar bucket y colas con AWS CLI.

Dueños: A (POM y `zones`), B (Keycloak), C (LocalStack).

Día 3 — `zones` completo

- [ ] Contrato `contracts/openapi/zones.yaml`.
- [ ] Migración Flyway `V1__create_zones.sql`.
- [ ] Dominio `Zone` y `ZoneRestriction` con validación.
- [ ] Casos de uso `CreateZoneUseCase` y `GetZoneUseCase`.
- [ ] Adaptador JPA y test con Testcontainers.
- [ ] Adaptador REST con `ProblemDetail`.
- [ ] Dockerfile multi-stage y servicio en Compose.
- [ ] `./mvnw verify` verde.

Dueño: A. **Revisión:** B y C.

Día 4 — Identidad y seguridad

- [ ] `identity` como fachada de usuarios y roles.
- [ ] Spring Security como *resource server* en `zones`.
- [ ] Conversor de roles de Keycloak.
- [ ] Pruebas 401 / 403 / 200.
- [ ] Documentar el flujo de token en el README del servicio.

Dueño: B. **Revisión:** A.

Día 5 — Eventos y CI

- [ ] Tabla `outbox_event` y relay con `SKIP LOCKED`.
- [ ] Publicación en EventBridge.
- [ ] Consumidor SQS de `history` con `processed_event`.
- [ ] Lambda `analytics-ingest` escribiendo en `raw/`.
- [ ] Prueba de idempotencia.
- [ ] Prueba de mensaje a DLQ.
- [ ] Workflow `.github/workflows/ci.yml` con build, tests, JaCoCo y Trivy.
- [ ] PR de Fase 0 mergeado con CI verde.

Dueño: C. **Revisión:** A y B.

Checklist de entorno por persona

- [ ] JDK 21 instalado y `java -version` correcto.
- [ ] Docker Desktop arrancado y `docker compose version` correcto.

- [ ] Repositorio clonado y `gh auth login` hecho.
- [ ] `.env` creado a partir de `.env.example`.
- [ ] `docker compose --profile infra up -d` funciona.
- [ ] Keycloak, Grafana y LocalStack accesibles.
- [ ] `./mvnw verify` ejecuta.

---

9. Semanas 2-3: primer flujo completo

Orden de implementación acordado:

- [ ] `zones` (hecho en Fase 0).
- [ ] `identity` (hecho en Fase 0).
- [ ] `user-profiles`.
- [ ] `inventory`.
- [ ] `availability`.
- [ ] `reservations`.
- [ ] `rides`.
- [ ] `history`.
- [ ] Lambda `analytics-ingest`.
- [ ] Primer mart y panel de Grafana.
- [ ] Colección Bruno del flujo de usuario.

Puerta de la semana 3: registro → reserva → viaje → fin → historial → evento en S3 → panel actualizado, en menos de 10 minutos y con un solo comando.

Reparto orientativo:

Persona	Semana 2	Semana 3
A	`user-profiles`, contrato y dominio de `reservations`	`reservations` completo, `rides`
B	`inventory`, máquina de estados de `availability`	Eventos de `availability`, pruebas de fallo
C	Lambda de ingesta, `history`	`history`, primer mart, Grafana, CI de integración

---

10. Rituales y definición de "hecho"

10.1. Semana tipo

Día	Ritual	Duración
Lunes	Planificación: 3 objetivos por persona	30 min
Miércoles	Integración parcial de lo terminado	45 min
Viernes	Integración completa + demo + acta	60 min

10.2. Definición de "hecho" por servicio

- [ ] Contrato OpenAPI en `contracts/openapi/`.
- [ ] JSON Schema de eventos publicados y consumidos.

- [ ] Migración Flyway.
- [ ] Dominio con tests unitarios.
- [ ] Adaptador con Testcontainers PostgreSQL.
- [ ] Consumidor/productor probado con LocalStack si aplica.
- [ ] Prueba de idempotencia si consume eventos.
- [ ] Test de arquitectura con ArchUnit.
- [ ] Dockerfile y servicio en Compose con healthcheck.
- [ ] README del servicio con arranque y ejemplos.
- [ ] `./mvnw verify` verde.
- [ ] PR revisado y mergeado con CI verde.

10.3. Reglas de PR

- [ ] Título con prefijo (`feat:` , `fix:` , `test:` , `docs:`).
- [ ] Descripción con qué, por qué y cómo se probó.
- [ ] Enlace a la issue.
- [ ] Menos de 400 líneas cambiadas siempre que sea posible.
- [ ] Una sola revisión obligatoria; dos si toca contratos o seguridad.

---

11. Riesgos y responsables

Riesgo	Impacto	Mitigación	Responsable	¿Aceptado?
Alcance excesivo	Alto	Recortar Fases 3-4; mínimos primero	Todos	☐
Docker/Testcontainers lentos en Windows	Medio	WSL2, `withReuse(true)` , perfiles	C	☐
Keycloak frena el arranque	Medio	Configuración una vez y plantilla común	B	☐
LocalStack no es AWS al 100 %	Medio	Diferencias documentadas; validar en Academy	C	☐
Contratos divergentes	Alto	Contratos primero y revisión cruzada	A	☐
Servicios acoplados por base de datos	Alto	Un usuario por base; revisión en PR	Todos	☐
Equipo descoordinado	Alto	Rituales fijos y tablero actualizado	A	☐
Semana perdida por AWS	Medio	AWS es opcional hasta Fase 5	Todos	☐

---

12. Primeras diez acciones

#	Acción	Dueño	Fecha límite	¿Hecha?
1	Validar las decisiones D1-D20	Todos	Día de la reunión	☐
2	Rellenar nombres en el reparto	Todos	Día de la reunión	☐
3	Decidir si se usan issues y crear las 5 de Fase 0	A	+1 día	☐
4	Escribir `.env.example`	C	+1 día	☐

#	Acción	Dueño	Fecha límite	¿Hecha?
5	Crear `docker-compose.yml` de infraestructura	C	+2 días	☐
6	Script de las 14 bases	C	+2 días	☐
7	POM padre y esqueleto de `zones`	A	+3 días	☐
8	Realm de Keycloak con roles y usuarios	B	+3 días	☐
9	Script de LocalStack (bucket, bus, colas, DLQ)	C	+3 días	☐
10	Primer PR de Fase 0 con CI verde	Todos	+5 días	☐

---

13. Acta de la reunión

Fecha: _____ Duración: _____ Asistentes: _____

13.1. Decisiones tomadas

ID	Decisión final	Observaciones
D1		
D2		
D3		
D4		
D5		
D6		
D7		
D8		
D9		
D10		
D11		
D12		
D13		
D14		
D15		
D16		
D17		
D18		
D19		
D20		

13.2. Decisiones aparcadas

Tema	Motivo	Revisar el

13.3. Acciones acordadas

Acción	Dueño	Fecha	Estado
			☐
			☐
			☐

13.4. Próxima reunión

Fecha: _____ Objetivo: _____

> Este documento no se firma: se actualiza. La versión válida es siempre la de `main`.